

MEMORIA TÉCNICA

EcoDrive

Arquitectura Polígota NoSQL para Movilidad Urbana Compartida

Sara García | Oscar Jiménez

Jordan Zuñiga | Gabriel Alexandru Iacob

Módulo: Bases de Datos NoSQL | Curso 2025-2026

Índice de Contenidos

1. Justificación Arquitectónica

- 1.1 MongoDB — Servicio A: Catálogo de Vehículos
- 1.2 Redis — Servicio C: Sesiones y Caché
- 1.3 Neo4j — Servicio D: Red Social de Carpooling

2. Scripts de Carga Detallados

- 2.1 Servicio A: MongoDB
- 2.2 Servicio C: Redis
- 2.3 Servicio D: Neo4j

3. Auditoría y Correcciones a la IA

- 3.1 MongoDB — Normalización encubierta
- 3.2 Redis — Uso pobre de estructuras
- 3.3 Redis — Ausencia de TTL
- 3.4 Neo4j — Antipatrón Nodos-Tabla

4. Resultados de Ejecución

1. Justificación Arquitectónica

EcoDrive es una plataforma de movilidad urbana compartida. En lugar de utilizar una única base de datos para todos sus servicios, adopta una **arquitectura polígloa**: cada servicio emplea el motor NoSQL que mejor se adapta a su modelo de datos y patrón de acceso. Este principio reconoce que no existe una base de datos óptima universal.

Tabla 1 Asignación de motores NoSQL por servicio

Servicio	Motor NoSQL	Razón principal
A — Catálogo Vehículos	MongoDB (Documental)	Esquema flexible: patinetes, bicis y coches tienen atributos completamente distintos
B — IoT / Telemetría	Cassandra (Columnar)	Omitido según enunciado
C — Sesiones y Caché	Redis (Clave-Valor)	Respuesta en microsegundos, TTL nativo para sesiones, operaciones atómicas
D — Red Social Carpooling	Neo4j (Grafos)	Las relaciones entre usuarios son ciudadanos de primera clase; JOINS SQL serían recursivos

1.1 MongoDB — Servicio A: Catálogo de Vehículos

Los vehículos de una flota compartida son **heterogéneos**: un patinete no tiene `tipo_combustible` y un coche no tiene `limite_peso_kg`. En SQL esto obligaría a columnas nulas o a una tabla por subtipo. MongoDB permite **documentos polimórficos**: cada vehículo solo lleva los campos que le corresponden.

Patrón de diseño: El historial de reparaciones se incrusta como array dentro del documento porque siempre se consulta junto al vehículo, nunca de forma independiente. Esto elimina la necesidad de un JOIN y garantiza que con una sola lectura se obtienen todos los datos.

1.2 Redis — Servicio C: Sesiones y Caché de Batería

La aplicación necesita conocer el nivel de batería de los vehículos y gestionar sesiones activas con **latencia de microsegundos**. Redis almacena todo en RAM, lo que hace imposible de replicar con un motor orientado a disco. Las tres características clave son:

- **HSET**: modela el estado del vehículo como campos independientes.
- **HINCRBY**: decrementa la batería de forma atómica, sin riesgo de condición de carrera.
- **EXPIRE**: elimina automáticamente las sesiones al terminar el viaje, sin ningún job de limpieza adicional.

1.3 Neo4j — Servicio D: Red Social de Carpooling

El caso de uso central es la **navegación por relaciones**: ¿quién va al mismo destino? ¿Quién tiene amigos en común? En SQL estas consultas requerirían JOINS recursivos $O(n^2)$ que se degradan con el tamaño de la red. Neo4j recorre arcos en **tiempo constante** independientemente del volumen de datos, y permite almacenar propiedades directamente en las relaciones (fecha, km, vehículo usado).

2. Scripts de Carga Detallados

2.1 Servicio A — MongoDB

El script se ejecuta desde `mongosh`. La primera instrucción selecciona la base de datos `ecodrive` y elimina la colección si ya existía, garantizando una carga limpia en cada ejecución.

```
db = db.getSiblingDB('ecodrive');
db.vehiculos.drop(); // limpieza previa
```

Se insertan tres vehículos en una sola operación mediante `insertMany`. Cada documento tiene el mismo esquema base (`id`, `tipo`, `marca`, `modelo`, `bateria_actual`, `ubicacion`, `historial_reparaciones`) y campos exclusivos según el tipo:

- **Patinete V-001:** campo propio `limite_peso_kg: 120`
- **Bicicleta V-002:** campos propios `num_marchas: 21` y `tipo_cambio: 'Shimano'`
- **Coche V-003:** campos propios `tipo_combustible`, `num_plazas`, `autonomia_km`

```
db.vehiculos.insertMany([
  { _id: 'V-001', tipo: 'patinete', marca: 'Xiaomi',
    bateria_actual: 87, limite_peso_kg: 120,
    ubicacion: { lat: 40.4168, lon: -3.7038, zona: 'Centro-Madrid' },
    historial_reparaciones: [
      { fecha: ISODate('2025-11-10'), pieza: 'freno trasero',
        tecnico: 'Carlos M.', coste: 15 }
    ] },
  { _id: 'V-002', tipo: 'bicicleta', marca: 'BH',
    bateria_actual: 63, num_marchas: 21, tipo_cambio: 'Shimano',
    historial_reparaciones: [
      { fecha: ISODate('2025-12-01'), pieza: 'cadena',
        tecnico: 'Laura P.', coste: 12 },
      { fecha: ISODate('2026-01-15'), pieza: 'rueda delantera',
        tecnico: 'Laura P.', coste: 35 }
    ] },
  { _id: 'V-003', tipo: 'coche', marca: 'Renault', modelo: 'Zoe',
    tipo_combustible: 'electrico', num_plazas: 5, autonomia_km: 395,
    historial_reparaciones: [
      { fecha: ISODate('2025-10-20'), pieza: 'bateria 12V auxiliar',
        tecnico: 'Taller Oficial Renault', coste: 180 }
    ] }
]);
```

Consultas incluidas:

La consulta 1 filtra por `$exists: true` sobre el campo `tipo_combustible`, que solo existe en coches. Esto demuestra la flexibilidad del esquema documental.

```
// Consulta 1 – solo coches (campo exclusivo)
db.vehiculos.find({ tipo_combustible: { $exists: true } },
  { _id:1, marca:1, tipo_combustible:1, bateria_actual:1 });

// Consulta 2 – disponibles con batería > 60%
db.vehiculos.find(
  { estado: 'disponible', bateria_actual: { $gt: 60 } },
  { tipo:1, marca:1, bateria_actual:1, 'ubicacion.zona':1 });

// Consulta 3 – historial completo de V-002
db.vehiculos.find({ _id: 'V-002' },
  { marca:1, modelo:1, historial_reparaciones:1 });
```

2.2 Servicio C — Redis

El script usa la convención de nombres jerárquica con `:` para organizar las claves. Se ejecuta línea a línea desde `redis-cli` (omitiendo las líneas de comentario que comienzan con `--`).

`FLUSHDB` limpia la base de datos activa antes de cargar datos nuevos. A continuación se crean tres tipos de estructuras:

```
FLUSHDB
```

```
-- Sesión de viaje: HSET + TTL de 60 segundos
HSET sesion:viaje:TKN-A1B2C3 usuario_id 101 vehiculo_id V-001 \
    inicio '2026-05-04T09:00:00' km_recorridos 0 estado activo
EXPIRE sesion:viaje:TKN-A1B2C3 60
TTL sesion:viaje:TKN-A1B2C3 -- respuesta: 60 (contador regresivo)

-- Caché estado batería (3 vehículos)
HSET vehiculo:V-001:estado bateria 87 ubicacion_zona 'Centro-Madrid' \
    estado disponible
HSET vehiculo:V-002:estado bateria 63 ubicacion_zona 'Retiro-Madrid' \
    estado disponible
HSET vehiculo:V-003:estado bateria 42 ubicacion_zona 'Vallecas-Madrid' \
    estado en_uso

-- Decrementar batería V-001 (atómico)
HINCRBY vehiculo:V-001:estado bateria -5 -- resultado: 82
HGET vehiculo:V-001:estado bateria -- verificación: '82'
HGETALL vehiculo:V-002:estado -- ver todos los campos

-- Sorted Set: vehículos disponibles ordenados por batería
ZADD vehiculos:disponibles:bateria 82 'V-001'
ZADD vehiculos:disponibles:bateria 63 'V-002'
ZREVRANGE vehiculos:disponibles:bateria 0 0 WITHSCORES -- top 1
```

Justificación de estructuras: La clave `vehiculo:V-001:estado` modela el estado del vehículo como un hash de campos independientes. Esto permite actualizar solo el nivel de batería (`HINCRBY`) sin leer ni reescribir el objeto completo. El Sorted Set `vehiculos:disponibles:bateria` mantiene los vehículos libres ordenados por score (= nivel de batería), de forma que `ZREVRANGE` devuelve instantáneamente el más cargado.

2.3 Servicio D — Neo4j

El script Cypher se ejecuta en el navegador de Neo4j o desde `cypher-shell`. `DETACH DELETE` elimina nodos y arcos sin errores de integridad referencial. Los nodos se crean primero, y las relaciones se establecen después con `MATCH + CREATE` para referenciar nodos ya existentes por su `id`.

```
// Limpieza
MATCH (n) DETACH DELETE n;

// Nodos Usuario
CREATE (u1:Usuario {id:'U-101', nombre:'Ana García', ciudad:'Madrid'})
CREATE (u2:Usuario {id:'U-202', nombre:'Borja Ruiz', ciudad:'Madrid'})
CREATE (u3:Usuario {id:'U-303', nombre:'Carmen López', ciudad:'Madrid'});

// Nodos Destino
CREATE (r1:Destino {id:'D-01', nombre:'Parque Tecnológico de Leganés',
                    tipo:'trabajo'})
CREATE (r2:Destino {id:'D-02', nombre:'Aeropuerto T4 Barajas',
                    tipo:'viaje'});

// Relaciones de amistad con fecha en el arco
MATCH (a:Usuario {id:'U-101'}), (b:Usuario {id:'U-202'})
CREATE (a)-[:CONOCE {desde: date('2024-03-15')}]>(b);
MATCH (a:Usuario {id:'U-202'}), (b:Usuario {id:'U-303'})
CREATE (a)-[:CONOCE {desde: date('2025-01-08')}]>(b);

// Relaciones de viaje con propiedades en el arco
MATCH (u:Usuario {id:'U-101'}), (d:Destino {id:'D-01'})
CREATE (u)-[:VIAJA_A {fecha:date('2026-05-04'), km:18,
                    vehículo_id:'V-003'}]>(d);
MATCH (u:Usuario {id:'U-202'}), (d:Destino {id:'D-01'})
CREATE (u)-[:VIAJA_A {fecha:date('2026-05-04'), km:22,
                    vehículo_id:'V-003'}]>(d);
MATCH (u:Usuario {id:'U-303'}), (d:Destino {id:'D-02'})
CREATE (u)-[:VIAJA_A {fecha:date('2026-05-04'), km:35,
                    vehículo_id:'V-001'}]>(d);
```

La consulta principal busca pares de usuarios que comparten destino y además se conocen. La condición `u1.id < u2.id` evita devolver el mismo par dos veces ($A \rightarrow B$ y $B \rightarrow A$). La consulta extra recorre la cadena `u1-amigo-u2` y filtra los casos donde `u1` y `u2` no tienen relación directa, generando sugerencias de carpooling.

```
// Consulta: amigos que van al mismo destino
MATCH (u1:Usuario)-[:VIAJA_A]>-(d:Destino)<-[:VIAJA_A]-(u2:Usuario)
WHERE (u1)-[:CONOCE]-(u2) AND u1.id < u2.id
RETURN u1.nombre AS viajero1, u2.nombre AS viajero2,
       d.nombre AS destino_compartido;

// Consulta extra: sugerencias de amistad por amigo en común
MATCH (u1:Usuario)-[:CONOCE]-(amigo:Usuario)-[:CONOCE]-(u2:Usuario)
WHERE NOT (u1)-[:CONOCE]-(u2) AND u1.id <> u2.id
RETURN u1.nombre AS usuario, u2.nombre AS sugerencia_amistad,
       amigo.nombre AS en_comun;
```


3. Auditoría y Correcciones a la IA

Los scripts iniciales se generaron mediante prompts dirigidos a Claude. A continuación se detallan los cuatro antipatrones detectados en las propuestas, el problema concreto que causa cada uno y la corrección aplicada. Esta sección constituye la justificación técnica para la defensa.

3.1 MongoDB — Normalización encubierta

Antipatrón detectado

La IA creó dos colecciones separadas (`vehiculos` y `reparaciones`) unidas por un campo `vehicle_id` , replicando el modelo relacional SQL dentro de MongoDB.

Problema para la defensa

MongoDB no tiene JOINS nativos eficientes entre colecciones. Cada vez que la app muestra un vehículo necesita hacer dos queries secuenciales: una para el vehículo y otra para sus reparaciones. Además, los dos documentos pueden quedar inconsistentes si una transacción falla a mitad.

Regla aplicada

"Incrusta cuando los datos siempre se leen juntos y no se comparten entre documentos."
Un historial de reparaciones solo pertenece a un vehículo y siempre se necesita con él.

Tabla 2 Comparativa: propuesta de la IA vs. corrección aplicada

Aspecto	Propuesta IA	Corrección aplicada
Estructura	<code>db.reparaciones.find({ vehicle_id: 'V-001' })</code>	<code>historial_reparaciones: [{...}, {...}]</code>
Queries necesarias	2 queries + posible inconsistencia	1 query atómica
Consistencia	Riesgo de datos huérfanos	Garantizada por diseño

3.2 Redis — Uso pobre de estructuras (String + JSON)

Antipatrón detectado

La IA almacenó el estado del vehículo como un String con JSON serializado (`SET vehiculo:V-001 '{"bateria":87,...}'`). Para decrementar la batería proponía `GET` → `deserializar` → `restar` → `SET`.

Problema para la defensa

La secuencia `GET-modificar-SET` **no es atómica**. Entre el `GET` y el `SET` otro proceso puede haber modificado el valor, provocando una condición de carrera: los datos finales serían incorrectos. Además se transfiere todo el objeto JSON para cambiar un solo número.

Corrección aplicada

HSET modela el vehículo como campos independientes. **HINCRBY** decrementa un campo en una operación atómica en el servidor Redis: ningún otro cliente puede interferir entre la lectura y la escritura.

Tabla 3 Comparativa: atomicidad en la actualización de batería

Aspecto	Propuesta IA (String JSON)	Corrección aplicada (Hash)
Comando	<code>SET vehiculo:V-001 '{"bateria":87,...}'</code>	<code>HSET vehiculo:V-001:estado bateria 87 estado disponible</code>
Decremento	<code>GET</code> → <code>parse</code> → <code>resta</code> → <code>SET</code> (4 pasos)	<code>HINCRBY ... bateria -5</code> (1 paso)
Atomicidad	No garantizada	Garantizada por Redis
Condición de carrera	Posible	Imposible

3.3 Redis — Ausencia de TTL en sesiones

Antipatrón detectado

La sesión de viaje se creó sin tiempo de expiración. La clave permanecería en memoria indefinidamente aunque el usuario cerrara la app.

Problema para la defensa

Redis es una base de datos en memoria RAM. Con miles de viajes diarios, las sesiones huérfanas acumularían megabytes de datos obsoletos, agotando la memoria disponible y degradando el rendimiento del sistema completo.

Corrección aplicada

Inmediatamente después de crear la sesión se ejecuta `EXPIRE`. Redis gestiona la expiración internamente sin ningún proceso externo de limpieza.

Tabla 4 Comparativa: gestión de sesiones con y sin TTL

Aspecto	Propuesta IA	Corrección aplicada
Comando de creación	<code>SET sesion:viaje:TKN-A1B2C3 'user:101,v:V-001,...'</code>	<code>HSET sesion:viaje:TKN-A1B2C3 usuario_id 101 vehiculo_id V-001 estado activo</code>
Expiración	Sin <code>EXPIRE</code> → clave permanente en RAM	<code>EXPIRE sesion:viaje:TKN-A1B2C3 60</code>
Impacto en memoria	Miles de sesiones → memoria agotada	Auto-eliminada a los 60s → memoria estable
Limpieza	Requiere job externo	Gestionada nativamente por Redis

3.4 Neo4j — Antipatrón Nodos-Tabla

Antipatrón detectado

La IA modeló las relaciones como nodos intermedios `:Viaje` para guardar fecha y km, creando la estructura `(Usuario)-[:PARTICIPA_EN]->(Viaje)-[:VA_A]->(Destino)`. Además, las relaciones de amistad no tenían dirección ni propiedades.

Problema para la defensa

En Neo4j las relaciones son ciudadanos de primera clase y pueden almacenar propiedades directamente. Crear nodos intermedios destruye la ventaja del recorrido de grafos: en lugar de recorrer un arco, el motor tiene que atravesar dos arcos y un nodo extra en cada consulta. Es replicar el modelo SQL de tabla de intersección.

Corrección aplicada

Las propiedades de una relación van en el arco. Se elimina el nodo `:Viaje` y se pone `fecha`, `km` y `vehiculo_id` directamente en `[:VIAJA_A {...}]`. Las relaciones de amistad usan `[:CONOCE {desde: date(...)}]` con dirección y propiedades.

Tabla 5 Comparativa: modelo de relaciones en Neo4j

Aspecto	Propuesta IA	Corrección aplicada
Modelo de viaje	<code>(u)-[:PARTICIPA_EN]->(v:Viaje {fecha, km})-[:VA_A]->(d)</code>	<code>(u)-[:VIAJA_A {fecha, km, vehiculo_id}]->(d)</code>
Arcos por relación	2 arcos + 1 nodo intermedio	1 arco directo
Amistad	<code>(u1)-[:AMIGO]-(u2)</code> (sin propiedades)	<code>(a)-[:CONOCE {desde:date(...)}]->(b)</code>
Complejidad de consulta	O(n) con nodos intermedios	O(1) recorrido de arco

4. Resultados de Ejecución

Todos los scripts fueron ejecutados en contenedores Docker locales. Los resultados reproducen exactamente la salida de cada motor.

Servicio A — MongoDB

```
{ acknowledged: true, insertedIds: { '0':'V-001', '1':'V-002', '2':'V-003' } } //
Insertados 3 vehiculos. // Consulta 1 - coches: { _id:'V-003', marca:'Renault',
modelo:'Zoe', tipo_combustible:'electrico', bateria_actual:42 } // Consulta 2 -
disponibles bateria > 60%: { _id:'V-001', tipo:'patinete', marca:'Xiaomi',
bateria_actual:87, ubicacion:{ zona:'Centro-Madrid' } } { _id:'V-002', tipo:'bicicleta',
marca:'BH', bateria_actual:63, ubicacion:{ zona:'Retiro-Madrid' } } // Consulta 3 -
historial V-002: historial_reparaciones: [ { fecha: ISODate('2025-12-01'),
pieza:'cadena', coste:12 }, { fecha: ISODate('2026-01-15'), pieza:'rueda delantera',
coste:35 } ]
```

Servicio C — Redis

Tabla 6 Resultados de comandos Redis ejecutados

Comando	Respuesta	Nota
FLUSHDB	OK	Base de datos limpiada
HSET sesion:viaje:TKN-A1B2C3 ...	5	5 campos creados
EXPIRE sesion:viaje:TKN-A1B2C3 60	1	TTL establecido con éxito
TTL sesion:viaje:TKN-A1B2C3	60	Contador regresivo activo
HSET vehiculo:V-00X:estado ...	3	3 campos por vehículo (x3)
HINCRBY vehiculo:V-001:estado bateria -5	82	87 - 5 = 82 OK
HGET vehiculo:V-001:estado bateria	'82'	Verificación correcta
ZADD vehiculos:disponibles:bateria 82 V-001	1	Añadido al sorted set
ZREVRANGE ... 0 0 WITHSCORES	['V-001','82']	Vehículo con más batería

Servicio D — Neo4j

Tabla 7 Resumen de nodos y relaciones creados

Tipo	Categoría	Cantidad
Nodo	:Usuario	3
Nodo	:Destino	2
Relación	:CONOCE	2
Relación	:VIAJA_A	3

Resultado consulta principal — amigos con destino compartido:

```
viajero1 | viajero2 | destino_compartido 'Ana García' | 'Borja Ruiz' | 'Parque Tecnológico de Leganés'
```

Resultado consulta extra — sugerencias de carpooling:

```
usuario | sugerencia_amistad | en_comun 'Carmen López' | 'Ana García' | 'Borja Ruiz' 'Ana García' | 'Carmen López' | 'Borja Ruiz'
```

Interpretación: Ana y Carmen no se conocen directamente pero comparten un amigo en común (Borja). El sistema genera una recomendación de carpooling basada en la proximidad de la red social y la coincidencia de destinos.